SCALABILITY AUDIT OF AN E-COMMERCE PLATFORM

ShopSphere — CSA06: Design and Analysis of Algorithms
CO1 – Assessment Tool 1

Student Name: vutla gayathri

Registration Number: 192110276

Department: Computer Science and Engineering

Date: _____

Note: This report is prepared from the supplied question bank. Review the derivations, fill in your details, and ensure you can explain the work during Viva Voce.

1. Objective

Audit the time and space efficiency of ShopSphere's back-end modules, derive asymptotic bounds, estimate operation counts for a catalogue of one million products, and recommend redesigns for a nightly processing window of 10 minutes.

2. Given Information and Assumptions

- Current catalogue size: n = 100,000; projected size: n = 1,000,000.
- Server capacity: 10^8 basic operations per second.
- Nightly processing deadline: 10 minutes = 600 seconds.
- M2 loop doubles i each iteration. Its exact count is derived from powers of two.
- For M4, leading-term models n^2 and $n \log_2 n$ are used for approximate operation estimates.
- For M5, a recursion-tree estimate is used with logarithm base 2 and T(1)=1.
- Estimated runtime is operations divided by 10^8; actual hardware runtime may differ.

3. Part (a) — Understand the Problem

Module	Input size	Dominant basic operation	Expected growth
M1 Duplicate detector	n products	Compare each product title/seller with every later product	$\Theta(n^2)$
M2 API rate limiter	n requests	Audit(Request[j]) in doubling-sized loops	$\Theta(n)$
M3 S1 sequential search	n products	Compare search key with each scanned product	Best $\Theta(1)$, average/worst $\Theta(n)$
M3 S2 binary search	n sorted index entries	Compare key with middle entry and halve search range	$\Theta(\log n)$
M4 recommendation engine	n customer-product records	Recursive calls plus linear merge	Current $\Theta(n^2)$; redesign $\Theta(n \log n)$
M5 sales analytics	n sales records	Recursive sorting work n log n per level	$\Theta(n \log^2 n)$

4. Part (b) — Iterative Modules

M1: Duplicate-Listing Detector

The outer loop chooses i from 0 through n−2. For each i, the inner loop compares it with n−i−1 later products. Therefore the exact number of comparisons is:

C(n) = (n−1) + (n−2) + … + 2 + 1 = n(n−1)/2.

Time complexity: Θ(n²), because the number of pairwise comparisons is quadratic. Auxiliary space: O(1), excluding storage of marked duplicate results and assuming marking does not allocate a new data structure.

At n = 1,000,000: C(n) = 1,000,000 × 999,999 / 2 = 499,999,500,000 comparisons.

M2: API Rate Limiter

The values of i are 1, 2, 4, 8, …, 2^(k−1), where k = ceil($\log_2$ n). The inner loop performs i Audit operations each iteration. Hence:

A(n) = 1 + 2 + 4 + … + 2^(k−1) = 2^k − 1 = 2^ceil($\log_2$ n) − 1.

Since n ≤ 2^ceil($\log_2$ n) < 2n (for n > 1), A(n) is Θ(n). The team member's Θ(n log n) claim is incorrect: the inner-loop sizes are not all n; they are a geometric sequence.

For n = 1,024, ceil($\log_2$ 1,024) = 10, so A(1,024) = 2^10 − 1 = 1,023 Audit operations.

Auxiliary space is O(1), because the loop uses only a fixed number of variables.

5. Part (c) — Substitution for S2

Given T(n) = T(n/2) + 1, with T(1) = 1.

T(n) = T(n/2) + 1

= T(n/4) + 1 + 1

= T(n/8) + 3

= T(n/2^k) + k

Set n/2^k = 1, so 2^k = n and k = $\log_2$ n.

T(n) = T(1) + $\log_2$ n = 1 + $\log_2$ n, for n = 2^k.

Thus the recurrence has Θ(log n) time complexity. An iterative binary search uses O(1) auxiliary space. A recursive binary search uses Θ(log n) stack space in the worst case.

6. Part (d) — Master Theorem

M4 current design: T(n) = 4T(n/2) + n

a = 4, b = 2, f(n) = n; n^(log_b a) = n^($\log_2$4) = n².

Since f(n)=n=O(n^(2−1)), f(n) is polynomially smaller than n². Master Theorem Case 1 applies, so T(n)=Θ(n²). Doubling n increases the leading quadratic work by approximately 4×.

M4 proposed redesign: T(n) = 2T(n/2) + n

a = 2, b = 2, f(n) = n; n^(log_b a) = n^($\log_2 2$) = n.

f(n)=Θ(n^(log_b a)), so Master Theorem Case 2 applies and T(n)=Θ(n log n). When n doubles, the leading estimate changes from n $\log_2 n$ to 2n($\log_2 n$+1), a factor of about 2($\log_2 n$+1)/$\log_2 n$, which approaches 2 for large n.

M5: T(n) = 2T(n/2) + n log n

Here a=2, b=2, n^(log_b a)=n and f(n)=n log n. The three basic Master Theorem cases, in their standard simple form, do not directly cover the extra log factor. The extended Master Theorem (or a recursion tree) gives T(n)=Θ(n $\log^2 n$).

Recursion-tree view: at level i, there are 2^i subproblems of size n/2^i. Their combined work is 2^i × (n/2^i) $\log_2$(n/2^i) = n($\log_2 n - i$). Summing over approximately $\log_2 n$ levels gives Θ(n $\log^2 n$).

7. Part (e) — Best, Average and Worst Search Cases

S1: Sequential search

- Best case: the product is first; 1 comparison.
- Worst case: the product is last or absent; n comparisons.
- If present, with each position equally likely, average successful comparisons = (n+1)/2.
- If the product exists with probability p and is absent with probability 1−p, expected comparisons E = p(n+1)/2 + (1−p)n.

For n = 1,000,000 and p = 0.5: E = 0.5(1,000,001/2) + 0.5(1,000,000) = 250,000.25 + 500,000 = 750,000.25 comparisons.

S2: Binary search

- Best case: the key equals the middle entry on the first comparison; 1 comparison.
- Average case: Θ(log n) comparisons for a reasonably distributed search target.
- Worst case: the search range is repeatedly halved until the key is found or the range is empty; Θ(log n) comparisons. At n=1,000,000, a conventional implementation needs at most about 20 comparisons.

Binary search requires the index to be sorted. The exact average comparison count depends on the distribution of successful and unsuccessful searches and on the implementation's midpoint rules.

8. Part (f) — Prove f(n) = Θ(n^2) and Not O(n)

Given f(n) = $3n^2 + 10n \log_2 n + 50$.

Lower bound: for n ≥ 1, f(n) ≥ $3n^2$. Choose $c_1 = 3$.

Upper bound: for n ≥ 1, $\log_2 n \le n$ and $50 \le 50n^2$. Thus f(n) ≤ $3n^2 + 10n^2 + 50n^2 = 63n^2$. Choose $c_2 = 63$ and $n_0 = 1$. Therefore $3n^2 \le f(n) \le 63n^2$ for all n ≥ 1, proving f(n)=Θ(n^2).

To show f(n) is not O(n), note f(n) ≥ $3n^2$. If f(n) ≤ cn for all n ≥ n_0, then $3n^2 \le cn$, so $3n \le c$. This cannot hold for arbitrarily large n. Hence f(n) is not O(n).

9. Part (g) — Scalability Decision

At n = 1,000,000, use the question's server rate of 100,000,000 basic operations per second. The table below provides leading-term estimates; recurrence constants and real implementation overheads may change measured values.

Module / model	Estimated operations	Estimated time
M1: n(n−1)/2	499,999,500,000	4,999.995 s ≈ 83.33 min — FAIL
M2: 2^ceil(log₂n)−1	1,048,575	0.0105 s — PASS
S2: binary search worst case	≈20 comparisons	≈0.0000002 s — PASS
M4 current: n² leading term	≈1,000,000,000,000	10,000 s ≈ 166.67 min — FAIL
M4 redesign: n log₂n	≈19,931,569	≈0.199 s — PASS
M5: n log₂n(log₂n+1)/2 + n	≈209,600,000	≈2.09 s — PASS
Report generator: 3n²+10n log₂n+50	≈3,000,199,315,736	≈30,002 s ≈ 500 min — FAIL

10. Recommendations

1. Redesign M1 first. Replace the all-pairs comparison with a hash-based grouping keyed by (seller, normalized product title). This can reduce expected work toward O(n), subject to hashing and normalization costs.
2. Adopt the proposed M4 redesign. Removing replicated four-way recursion changes the asymptotic bound from Θ(n²) to Θ(n log n).
3. Optimize the report generator. Its 3n² term dominates; identify and eliminate the quadratic operation if possible.
4. M2, S2, and M5 meet the 10-minute window under the stated estimates, though production profiling is still recommended.
5. Benchmark with representative data and measure memory, database/network costs, and tail latency before deployment.

11. Source Code and Execution Verification

The Python program computes the estimated operation counts and runtimes using the question's server capacity. Run it from the repository root with:

```
python Question_1/Source_Code/shopsphere_audit.py
```

Figure 1. Source code screenshot:

Source Code — shopsphere_audit.py

```
"""
ShopSphere Scalability Audit
Course: CSA06 - Design and Analysis of Algorithms
CO1 - Assessment Tool 1

Implements the main numerical estimates required by the question bank.
The calculations are estimates based on the stated asymptotic models;
actual runtime depends on hardware, implementation, and data.
"""

from math import ceil, log2


def runtime_seconds(operations, ops_per_second=10**8):
    return operations / ops_per_second


def format_time(seconds):
    if seconds < 1:
        return f"{seconds:.9f} s" if seconds >= 0.000001 else f"{seconds:.3e} s"
    if seconds < 60:
        return f"{seconds:.3f} s"
    return f"{seconds / 60:.2f} min"


def main():
    n = 1_000_000
    ops_per_second = 10**8
    log_n = log2(n)

    # M1: every pair is compared once.
    m1 = n * (n - 1) // 2

    # M2: doubling loop; exact count is 2**ceil(log2(n)) - 1.
    m2 = 2 ** ceil(log2(n)) - 1

    # S2: conventional binary search on one million entries.
    # Maximum comparisons for a standard binary search at this n is 20.
    s2_worst = ceil(log2(n + 1))

    # M4 operation estimates use the leading terms of their recurrences.
    m4_current = n ** 2
    m4_redesign = int(round(n * log_n))

    # M5: recursion-tree estimate for T(n)=2T(n/2)+n log2(n).
    # For n=2**k and T(1)=1, T(n)=n*k*(k+1)/2+n.
    # Use log2(n) as a continuous estimate for n=1,000,000.
    m5 = n * log_n * (log_n + 1) / 2 + n

    # Report generator from the question bank.
    report_generator = 3 * n**2 + 10 * n * log_n + 50

    rows = [
        ("M1 Duplicate-listing detector", m1),
        ("M2 API rate limiter", m2),
        ("S2 Binary search (worst-case estimate)", s2_worst),
        ("M4 Current recommendation engine (leading term)", m4_current),
        ("M4 Redesigned recommendation engine (leading term)", m4_redesign),
        ("M5 Sales analytics engine (recursion-tree estimate)", m5),
        ("Nightly report generator f(n)", report_generator),
    ]

    print("SHOPSPHERE SCALABILITY AUDIT")
    print("=" * 72)
    print(f"Catalogue size n: {n:,}")
    print(f"Server capacity: {ops_per_second:,} basic operations/second")
    print()
    print(f"{'Module':56} {'Operations':>18} {'Estimated time':>16}")
    print("-" * 94)
    for label, operations in rows:
        print(f"{label:56} {operations:18,.2f} {format_time(runtime_seconds(operations)):>16}")
    print()
    print("M2 verification for n = 1,024:")
    print(f"  Exact Audit operations = 2^ceil(log2(1024)) - 1 = {2 ** ceil(log2(1024)) - 1:,}")
    print("  Therefore M2 is Theta(n), not Theta(n log n).")
    print()
    print("Decision for a 10-minute nightly window:")
    print("  M1: FAIL (about 83.33 minutes)")
```

Figure 2. Output screenshot from executing the program:

Executed Program Output

```
SHOPSPHERE SCALABILITY AUDIT
======================================================================
Catalogue size n: 1,000,000
Server capacity: 100,000,000 basic operations/second

Module                                                    Operations   Estimated time
-------------------------------------------------------------------------------------------
M1 Duplicate-listing detector                        499,999,500,000.00        83.33 min
M2 API rate limiter                                        1,048,575.00    0.010485750 s
S2 Binary search (worst-case estimate)                            20.00      2.000e-07 s
M4 Current recommendation engine (leading term)      1,000,000,000,000.00       166.67 min
M4 Redesigned recommendation engine (leading term)          19,931,569.00    0.199315690 s
M5 Sales analytics engine (recursion-tree estimate)        209,599,497.10          2.096 s
Nightly report generator f(n)                        3,000,199,315,735.69       500.03 min

M2 verification for n = 1,024:
  Exact Audit operations = 2^ceil(log2(1024)) - 1 = 1,023
  Therefore M2 is Theta(n), not Theta(n log n).

Decision for a 10-minute nightly window:
  M1: FAIL (about 83.33 minutes)
  M2: PASS
  S2: PASS
  M4 current design: FAIL (about 166.67 minutes under the n^2 leading-term model)
  M4 redesigned design: PASS
  M5: PASS under the stated recursion-tree estimate
  Report generator: FAIL (quadratic leading term; about 500 minutes)

Recommendation: replace M1's pairwise scan with a hash-based grouping/index
on (seller, normalized title); remove M4's replicated four-way recursion;
optimize the report generator to avoid its quadratic term.
```

Figure 3. Analysis summary:

Analysis Summary

```
SHOPSPHERE — KEY ANALYSIS
M1: Comparisons = n(n−1)/2; Θ(n²); auxiliary space O(1).
M2: Audits = 1+2+4+…+2^(k−1) = 2^k−1, where k=ceil(log₂ n); Θ(n).
S2: T(n)=T(n/2)+1; for n=2^k, T(n)=log₂ n+1; Θ(log n).
M4 current: 4T(n/2)+n → Θ(n²). Redesigned: 2T(n/2)+n → Θ(n log n).
M5: 2T(n/2)+n log n → Θ(n log² n).
Report generator: 3n²+10n log₂n+50 → Θ(n²), and not O(n).
Redesign priorities: M1, current M4, and the quadratic report generator.
```

12. Conclusion

The audit identifies M1, the current M4 design, and the quadratic report generator as the main scalability risks for one million products. M2 is linear despite the original Θ(n log n) claim. S2 and the redesigned M4 are substantially more scalable, while M5 grows as Θ(n log²n) and remains within the estimated nightly window. These are theoretical projections and should be validated by benchmarking.

13. Documentation / Academic Integrity

I _____ (Name / Registration No.) certify that this submission is my original work and that I have followed the assessment guidelines. I understand that violations of academic integrity rules may result in disciplinary action.

Student Signature: _____ Date: _____